

Practical Secure Inference Algorithm for a Fine-Tuned Large Language Model Based on Fully Homomorphic Encryption

Ruoyan Zhang^{1b}, Zhongxiang Zheng^{1b}, and Wankang Bao

Abstract—Large language models (LLMs) are currently at the forefront of the machine learning field, showing broad application prospects but at the same time presenting certain risks of privacy leakage. Both the training datasets and the user's input data during interactions face security issues, which need to be addressed urgently before their further development. To address this problem, we combine privacy-preserving techniques such as fully homomorphic encryption (FHE) and provable security theory with parameter-efficient fine-tuning (PEFT) to propose an efficient and secure inference scheme for LLMs that protects both the user-side's input and the server-side's private parameters. More specifically, we focus on pretrained LLMs that rely on open-sourced base models and then are fine-tuned with private datasets by LoRA. This is a popular roadmap for vertical domain large models such as LawGPT and BenTsao. To achieve this efficient and secure inference LLM scheme, we use two key technologies that are summarized below: 1) we divide the whole model into two parts, denoted as the public part and the private part. The weights of the public part are publicly accessible (e.g., the open-sourced base model), whereas those of the private part need to be protected (e.g., the LoRA matrices). Then, the public part is deployed on the client side, and the server maintains the private part. In this way, the overhead associated with computing on private data can be greatly reduced. 2) we propose a general method to transform a linear layer into another one that provides security against model extraction attacks and preserves its original functionality, denoted as the private linear layer (PLL). Afterwards, we use this method on the LoRA matrices of the server side, where the PLL changes the computation of the LoRA matrices in a way that accomplishes correct inference and ensures that the server protects their private weights without restricting the user's input. We also show that the difficulty of performing model extraction attacks for the PLL can be reduced to the well-known hard problem of learning with errors (LWE). Combining this method with FHE, we can obtain an inference algorithm for fine-tuned LLMs that protects the user's input and the server's private weights at the same time. In this paper, we use the open-source model ChatGLM2-6B as the base model, which is fine-tuned by LoRA. The experimental results show that the inference efficiency of our scheme reaches 1.61 s/token, demonstrating that the scheme is highly practical.

Index Terms—Cryptography, application security, cryptographic protocols, generative pre-trainer transformer.

Received 24 January 2025; revised 24 September 2025 and 20 November 2025; accepted 22 November 2025. Date of publication 3 December 2025; date of current version 23 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62202437, Grant 12271306, Grant 62302467, and Grant 62201525; and in part by the National Cryptologic Science Fund of China under Grant 2025NCSF02018. The associate editor coordinating the review of this article and approving it for publication was Dr. Arcangelo Castiglione. (Corresponding author: Zhongxiang Zheng.)

The authors are with the Communication University of China, Beijing 100024, China (e-mail: zhengzx@cuc.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3639994

I. INTRODUCTION

CHATGPT sparked heated debates as soon as it was launched in 2022 and has been in the public eye thus far because of its powerful language understanding and generation with coherent and logical contextual semantics. It is now a hot topic for researchers and a direction of development for companies. It has long been integrated into people's lives as a tool that has changed modern human-computer interactions. Such large language models (LLMs) have shown broad application prospects; however, they also expose the risk of privacy leakage.

With respect to accurate responses to common sense questions, all machine learning models display varying degrees of memorization phenomena [1]. Similarly, LLMs tend to memorize the underlying training data directly. When given the top text of a training dataset, LLMs have a high probability of directly outputting the following text of that data in the training dataset; e.g., Carlini et al. used this method to extract 600 pieces of training data from GPT-2 [2]. Their demonstration revealed that the training data are less secure and that this memorization enables an attacker to extract the underlying training data by posing questions wherever they have access to the model.

In accordance with the privacy policy of OpenAI [3], the company that developed ChatGPT, they collect personal information that is included in the inputs, file uploads, or feedback and use this information to improve their services and conduct research. This suggests that users' input data may also enter the training set and face the extraction problem described above in practice. Because current terminal devices still cannot easily deploy LLMs locally, private inputs must be transmitted to the server when users interact. These private data can easily be intercepted by others, recorded by the server and used for training purposes if they are not encrypted.

To ensure the security of the input data, privacy-preserving computing techniques for small deep learning models such as deep neural networks (DNNs) and convolutional neural networks (CNNs) have been extensively studied. For example, in 2018, Badawi et al. proposed a CNN based entirely on fully homomorphic encryption (FHE) that is able to homomorphically classify encrypted images [4]. Their solution achieved a sufficient security level and reasonable classification accuracy. However, such research on large models is currently in its initial stages. Currently, most large models are based on the transformer model [5], which is a neural network architecture based on self-attention and is widely used in natural language

processing tasks. Nevertheless, few studies exist on privacy-preserving techniques for the transformer model.

In 2022, Chen et al. proposed THE-X [6], a privacy-preserving inference method for transformers using FHE with polynomial approximations. However, accurate and efficient approximations of nonlinear functions remain challenging as the model size increases [4]. In the same period, Iron [7] introduced secure two-party protocols for nonlinear operations, followed by BumbleBee [8] and CipherGPT [9], which extended such protocols to handle operations such as softmax, GELU, and LayerNorm, although these systems were limited to relatively small models. Zama's Concrete ML [10] further explored an FHE-based client/server protocol for LLM inference, delegating nonlinear layers to the client while executing linear layers over encrypted data on the server. Gong et al. also presented a multimodal federated learning framework [11] based on homomorphic encryption, enabling secure cross-modal feature fusion without relying on a trusted third party. More recent works in 2024 advanced both efficiency and security. SecureGPT [12] extended CipherGPT with more efficient MPC-based protocols, improving performance but remaining constrained to GPT-2 scale models. Rovida and Leporati [13] proposed an FHE-based circuit for BERT-tiny, leveraging CKKS with bootstrapping and pre-computed LayerNorm to enable encrypted sentiment analysis with approximately 15 s/token. Zhao et al. introduced Silent Guardian [14], a defence mechanism for 7B-scale LLMs that mitigates malicious exploitation via truncation protection and tailored perturbations. In 2023, Dong et al. proposed PUMA [15], an MPC-based framework enabling lossless inference of large models by designing high-precision operators, achieving 200 s/token on LLaMA-7B. Additionally, building on MPC, Rathee et al. proposed an MPC-minimized secure LLM inference framework MARILL [16], which limits MPC to the top layers and applies LoRA-based low-rank adaptation, demonstrating feasibility on open-llama3b-v2.

Another common method to improve the efficiency of ciphertext computations is to use homomorphic encryption to circumvent complex with transmissions. The complex computations are passed back to the data owner to be completed in the plaintext state. Using this idea, Lam et al. proposed a hybrid PE-NN model based on FHE in 2023 [17], where a pretrained open-source image prediction CNN model is followed by an additional linear network. The CNN model is subsequently deployed on the client side, and the linear network is deployed on the server side. The client uses the local open-source network to compute the input and then encrypts and transmits the encrypted data to the server. The server returns an encrypted predicted output through the private network. This method guarantees that user data are transmitted in ciphertext while avoiding the need for nonlinear operations to be performed using private computations via FHE. By substituting computationally intensive ciphertext evaluations on the server side with low-cost plaintext computations on the client side, this approach effectively reduces the overhead associated with private computations. However, the model's capability is bounded by the ability of the CNN because the private part is made up of a linear network, which does not

improve the ability of the CNN model to address problems. In addition, a linear network is weak in terms of security against model extraction attacks; an attacker can calculate the model weights by modifying the input data and observing changes in the output results, which poses a significant security risk.

To avoid these problems, we focus on the low-rank adaptation (LoRA) [18] technique for fine-tuning LLMs. To flexibly adapt models to the specific task requirements of a vertical domain, fine-tuning on pretrained models has become a paradigm for dealing with natural language processing tasks, which can lead to very large performance gains on specific tasks [19], [20], [21]. However, as models grow in size, fine-tuning all the model parameters on standard consumer hardware becomes impractical. To address this problem, an efficient fine-tuning technique called parameter-efficient fine-tuning (PEFT) has become popular. PEFT can fix most of the pretrained model parameters and fine-tune only a small number of additional model parameters. Therefore, PEFT significantly reduces computational and storage costs while achieving performance comparable to that of full-parameter fine-tuning. Common local fine-tuning techniques include prefix tuning [22], prompt tuning [23], and LoRA [18]. Among them, the LoRA fine-tuning technique is among the most versatile and the most effective fine-tuning methods. More specifically, LoRA freezes the pretrained model weights and injects trainable bypass matrices into each layer of the transformer architecture. These bypass matrices are used to simulate full-parameter fine-tuning based on the intrinsic low-rank nature of large models, and the use of two low-rank matrices in the bypass matrices reduces the number of parameters that need to be tuned in the model from $d * d$ to $2 * r * d$ ($r \ll d$) without changing the dimension of the output data. Finally, when saving the fine-tuning weights, one only needs to save the portion of the low-rank matrices. In this way, LLMs with LoRA will not change the architecture of the original model and will not produce a delay in inference.

Combining privacy-preserving technologies with LoRA, we propose an efficient and secure inference scheme for LLMs with LoRA based on FHE. We deploy the base LLM on the client and the low-rank matrices obtained by fine-tuning on the server. In the inference process, the user encrypts the data when it needs to pass through the low-rank matrix section and sends it to the server; then, the server completes ciphertext computation before the ciphertext result is returned. Finally, the user decrypts the result to finish the inference. This method can greatly reduce the overhead associated with computing on private data and improve the inference efficiency.

In this process, an important security issue is how to protect the private weights on the server side because the low-rank matrix is a simple linear layer, and the parameters of this part can be easily computed by an attacker using Model Extraction Attacks. To address this problem, we introduce a general transform method that can be applied to any linear layer to gain extra protection against model extraction attacks known as the private linear layer (PLL). We show that the PLL can improve the security of the original linear layer and preserve its original functionality. We also show that the difficulty of performing

model extraction attacks for the PLL can be reduced to the well-known hard problem LWE.

By combining these techniques, our solution can thoroughly minimize ciphertext computations to achieve high efficiency and simultaneously protect the security of the user's input on the client side and the private weights on the server side. The article is organized as follows. In Section II, we introduce the mathematical notation and related techniques utilized in this paper. We propose the architecture of our scheme with its security analysis in Section III. In Section IV, we present the experimental results of our scheme, including a comparison with former methods. Finally, we summarize our contributions and discuss potential future work in Section V.

II. BACKGROUND

A. Notations

The following is a description of the symbols used in this paper:

- $\mathbb{R}$. The set of real numbers. The matrix with dimension $m \times n$ belonging to set $\mathbb{R}$ is denoted as $\mathbb{R}^{m \times n}$.
- $x \sim D$. The sample from distribution D yields x .
- $\mathbb{Z}_q^m$. The integer matrix ring with dimension m , where addition, subtraction and multiplication are performed modulo q .
- $R = \mathbb{Z}[X]/(X^N + 1)$. For a power-of-two integer N , the cyclotomic polynomial ring of dimension N .
- $x \leftarrow F$. Assigning the result of function F to x .
- $Enc(m)$. Encrypt plaintext m .
- $Dec(ct)$. Decrypt ciphertext ct .
- $Add(ct_1, ct_2)$. Return the sum of ct_1, ct_2 in ciphertext.
- $CMult(ct; c)$. For ciphertext $ct_1 = Enc[a_1, a_2, \dots, a_n]$ and plaintext $c = [b_1, b_2, \dots, b_n]$, return the ciphertext $ct_2 = Enc[a_1 * b_1, a_2 * b_2, \dots, a_n * b_n]$.
- $Mult(ct_1; ct_2)$. For ciphertext $ct_1 = Enc[a_1, a_2, \dots, a_n]$ and $ct_2 = Enc[b_1, b_2, \dots, b_n]$, return the ciphertext $ct_3 = Enc[a_1 * b_1, a_2 * b_2, \dots, a_n * b_n]$.
- $Rescale(ct; p)$. For ciphertext ct and integer p , reduce the level of ciphertext while reducing noise.
- $Rotate(ct; r)$. Return a ciphertext encrypting the rotated plaintext vector of ct by r positions.

B. Transformer Model

The Seq2Seq model [24] is an important architecture in natural language processing that is designed to handle variable-length input and output sequences through its encoder-decoder structure. The encoder compresses the input sequence into a fixed-dimensional information vector, which is then decoded by the decoder to generate the output.

In traditional RNN-based Seq2Seq models, both the encoder and decoder are recurrent networks. The final state of the encoder contains the entire input information and serves as the decoder's initial state. However, as Cho et al. [25] demonstrated, RNNs tend to forget information in long sequences, leading to incorrect outputs. Additionally, their sequential nature prevents parallelization.

To address these limitations, Vaswani et al. [5] proposed the Transformer architecture in 2017. This architecture employs

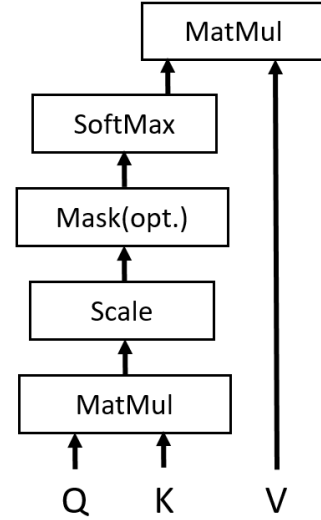


Fig. 1. Self-attention in the transformer architecture.

self-attention mechanisms and feed-forwards networks instead of recurrence, enabling parallel computation and better handling of long-range dependencies. The encoder consists of multihead attention and feed-forward sublayers, while the decoder includes an additional masked multihead attention layer. Each sublayer is followed by an Add & Norm operation, where Add denotes residual connections and Norm denotes layer normalization.

Self-attention, a core component, effectively captures the dependencies between different sequence positions. An overview of its structure is illustrated in Fig. 1. Specifically, as shown in Fig. 2a, the input matrix X is linearly transformed using trainable matrices W^Q , W^K , and W^V to obtain the Query (Q), Key (K), and Value (V) vectors. The attention output is computed as follows:

$$Attention(Q, K, V) = softmax(Q \cdot K^T \cdot M) \cdot V$$

where M can be considered the deviation matrix. LLMs stack multiple such layers, resulting in models with billions of parameters, which are primarily contained in self-attention modules.

C. LoRA (Low-Rank Adaptation)

Fine-tuning, a central topic in AI model optimization, is among the most common practices in transfer learning. For models with fewer than one million parameters, full-parameter fine-tuning remains manageable. However, with the emergence of large-scale language models such as GPT, which contain billions of parameters, full fine-tuning has become increasingly memory intensive and slow. In this context, PEFT methods have gained significant importance.

LoRA, a PEFT technique introduced by Microsoft Research in 2021 [18], is among the most versatile and effective approaches. It reduces the number of trainable parameters by applying low-rank decomposition to the weight matrices of the LLM, thereby lowering the computational and memory requirements. For instance, compared with full fine-tuning of

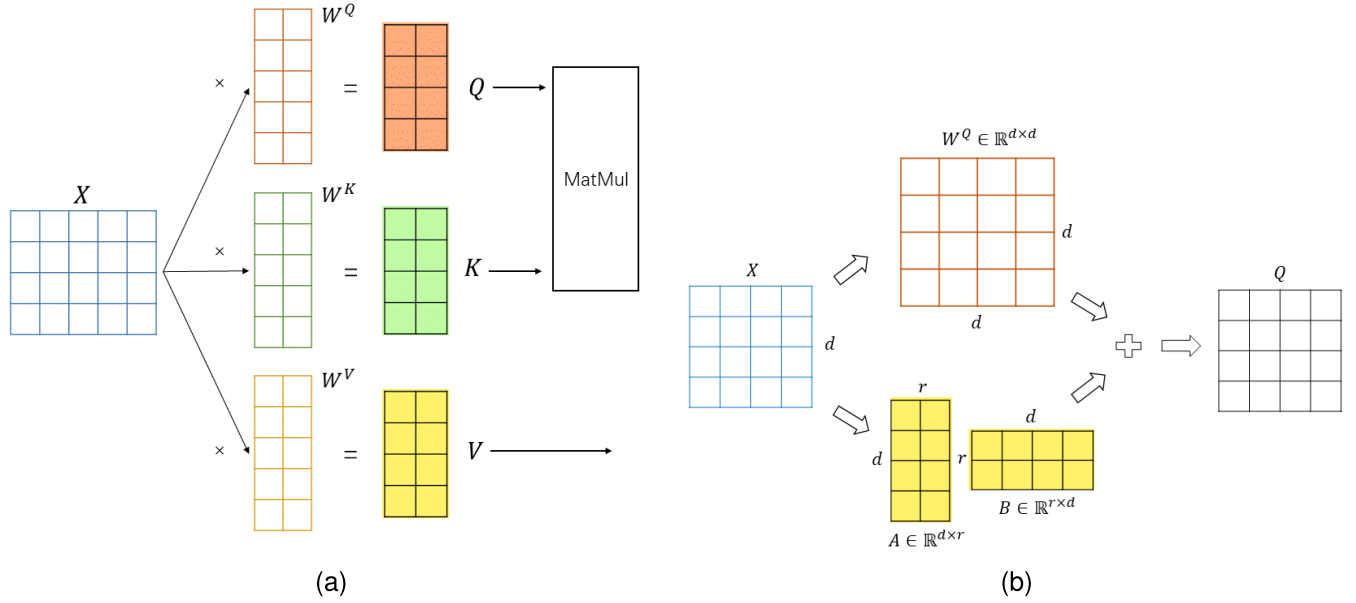


Fig. 2. Illustrations of the key components. (a) Calculation of Q , K , and V in self-attention. (b) LoRA applied to the transformer with $d = 4$, $r = 2$.

GPT-3 175B with Adam, LoRA reduces trainable parameters by 10,000 times and GPU memory usage by a factor of 3 [18].

Inspired by the observation that overparameterized models exhibit low intrinsic dimensionality, LoRA posits that weight updates during fine-tuning also possess low rank. Specifically, for a weight matrix $W_{d \times d}$ and its update $\Delta W_{d \times d}$, we have the following:

$$\text{rank}(\Delta W_{d \times d}) \ll d$$

1) *Low-Rank Decomposition*: Given a pretrained weight $W \in \mathbb{R}^{d \times d}$ and an incremental update $\Delta W \in \mathbb{R}^{d \times d}$, the output is computed as $h = Wx + \Delta Wx$. LoRA approximates ΔW using two low-rank matrices:

- $A \in \mathbb{R}^{d \times r}$: randomly initialized with Gaussian values,
- $B \in \mathbb{R}^{r \times d}$: initialized to zero.

Thus, $\Delta W \approx AB$, reducing the parameter count from d^2 to $2rd$ (with $r \ll d$), yields the following:

$$h = Wx + ABx.$$

A scaling hyperparameter α is applied to stabilize the training, resulting in the following final form:

$$h = Wx + \frac{\alpha}{r} ABx.$$

2) *Applied to the Transformer*: LoRA can be applied to any weight subset in a network. In the transformer architecture, the self-attention module contains four weight matrices (W^Q , W^K , W^V , and W^O), while the MLP module has two. For efficiency, LoRA typically freezes the MLP weights and applies low-rank adaptation only to the self-attention matrices—e.g., W^Q is augmented with bypass matrices, as illustrated in Fig. 2b.

D. Homomorphic Encryption

Homomorphic encryption is a cryptographic technique based on the computational complexity theory of mathematical problems. This idea was first proposed by Rivest [26] in 1978.

The term “homomorphic” means that after homomorphic encryption, the result of a computation on the ciphertext is the same as the result of a direct computation on the plaintext after decryption. Let F be a function that performs computations on x and y , E be the encryption function and D be the decryption function; then, we have the following:

$$D(F(E(x), E(y))) = F(x, y).$$

Homomorphic encryption methods can be divided into three categories: partially homomorphic encryption (PHE), somewhat homomorphic encryption (SHE) and fully homomorphic encryption (FHE). PHE and SHE have several limitations in terms of the types and quantities of ciphertext calculations. FHE supports both the addition and multiplication operations of ciphertext, which can be further expanded to arbitrary computations according to the logical completeness.

In 2009, Gentry [27] proposed an FHE scheme using ideal lattices, which represents the first theoretically feasible blueprint of FHE. Since then, three generations of technical systems have been developed for FHE. The CKKS FHE algorithm, proposed by Cheon et al. [28] in 2017, is a second-generation FHE algorithm that is especially suitable for efficient floating-point approximation calculations. In addition, the CKKS algorithm employs a rescaling technique to keep the message sizes before and after encoding essentially unchanged during the homomorphic computation, which is currently the most efficient FHE algorithm for numerical computation and can be implemented using the Microsoft SEAL library [29] developed by Microsoft’s Cryptography and Privacy Research Group.

E. Learning With Errors

The learning with errors (LWE) problem was introduced by Regev in 2005 [30] and has been widely used in the construction of cryptographic schemes. LWE is proven to be

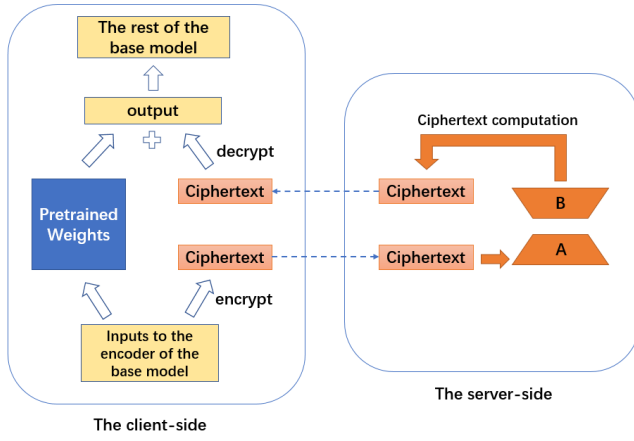


Fig. 3. Data transmission between the client and server during self-attention of the transformer.

difficult since there is a reduction from LWE to lattice-based hard problems. There are two versions of LWE problems, namely, the search version and the decisional version, which are defined as follows:

Definition 1 (Search-Version LWE (LWE)): Given n samples following the LWE distribution, that is, $\{(A \in \mathbb{Z}_q^{m \times n}, b = sA + e \bmod q)\}$, where $A \sim U(\mathbb{Z}_q^{m \times n})$ is randomly selected and $s, e \sim \chi$. The goal of search-LWE is to find s .

Definition 2 (Decisional LWE (DLWE)): Given n samples, distinguish whether they follow the LWE distribution $\{(A \in \mathbb{Z}_q^{m \times n}, b = sA + e \bmod q)\}$ or uniform distribution $\{(A \in \mathbb{Z}_q^{m \times n}, b \in U(\mathbb{Z}_q^n))\}$.

Many variants have been explored since the introduction of LWE. The continuous LWE (CLWE) problem was proposed by Bruna et al. in 2021 [31] and can be seen as a continuous variant of the LWE problem. Bruna et al. also proved that CLWE can be reduced to LWE under certain conditions.

Definition 3 (CLWE): Let m be an integer, and let $\gamma \geq 2\sqrt{m}$, $\beta \in (0, 1)$. Given n samples $\{(A \in \mathbb{R}^{m \times n}, b = \gamma sA + e \bmod 1 \in \mathbb{R}^n)\}$, where the secret vector $s \in \mathbb{R}^m$ is of length 1, the elements in A follow the standard normal distribution, i.e., $N(0, 1)$ and e follows $N(0, \beta)$. The CLWE problem involves finding the secret vector s .

III. PRACTICAL SECURE INFERENCE ALGORITHM FOR A FINE-TUNED LARGE LANGUAGE MODEL BASED ON FHE

A. Open-LLM + Private-LoRA

On the basis of whether private data participate in training, we decompose the weights involved in LLM inference into modular components and propose an “Open-LLM + Private-LoRA” architecture. “Open-LLM” denotes the open-source, pretrained base model that delivers fundamental semantic understanding; its parameters remain fully public. “Private-LoRA” refers to the rank-decomposition bypass matrices fine-tuned with private data and injected into the base model to encode specialized knowledge.

This decomposition clarifies that “Open-LLM” never touches private data, so it can be distributed to end users and executed locally in plaintext. In contrast, “Private-LoRA”

is trained on confidential data; its low-rank parameters must remain protected and therefore reside on the training server. Users interact with this server during inference. To prevent input leakage, users encrypt their queries before transmission, and the server operates entirely on ciphertext. Under this workflow, model inference is split into two stages: one executed in plaintext and the other involving ciphertext. By constraining the ciphertext stage to only the necessary weights, we enhance the efficiency of the encrypted inference algorithm. Figure 3 shows the structural decomposition and encrypted components.

Initialization proceeds as outlined in Figure 4a. Let M denote the base LLM, $\text{FineTune}(M)$ denote the LoRA fine-tuning applied to M , and L denote the resulting low-rank matrices.

During inference, whenever computation traverses the fine-tuned components, the interaction pattern in Figure 4b applies. The user input is x , whereas $\text{PInf}_1(M, x)$ and $\text{PInf}_2(M, x)$ denote the base model computations on x before and after the fine-tuned segment, respectively. $\text{LoRA}_{\text{FHE}}()$ represents the evaluation of c_L under fully homomorphic encryption.

B. Private Linear Layer

In this section, we present a general procedure for converting any linear layer into a variant that retains its functionality while improving its resilience against model-extraction attacks. We begin by formalizing the threat.

1) *Attack on a Plain Linear Layer:* Let the layer weights be a matrix $A \in \mathbb{R}^{m \times n}$. For an input $X \in \mathbb{R}^{d \times m}$, the output is $Y = XA \in \mathbb{R}^{d \times n}$. Consider an adversary who can submit multiple queries with inputs of their choice. Without loss of generality, set $d = 1$. If the attacker queries $x_1 = (1, 0, \dots, 0), \dots, x_n = (0, 0, \dots, 1)$, they can recover all the columns of A in $O(n)$ queries. Therefore, a plain linear layer is intrinsically vulnerable to model extraction.

2) *Proposed Defence:* We mitigate this weakness in two stages. First, we replace the linear layer with

$$y = (x \ x') \begin{pmatrix} A \\ E' \end{pmatrix} + sA \bmod q,$$

where $x' \in \mathbb{R}^{d \times m'}$, $E' \in \mathbb{R}^{m' \times n}$, $s \in \mathbb{R}^{d \times m}$, and $q \in \mathbb{R}$.

Specifically, every entry of x' is set to 1; s comprises d vectors sampled independently from the uniform distribution over vectors of length γ in $\mathbb{R}^m$; and $q > 0$ is a fixed real constant. During training, we choose γ and $q > 0$, initialize each entry of E' from a small Gaussian, and keep x' , s , and q fixed throughout.

After convergence, we store the last l checkpoints, denoted (A_i, E'_i) for $i \in \{1, \dots, l\}$. Because the dimensions of A_i and E'_i scale linearly with the number of LoRA parameters, storing these l pairs requires little additional space. The second stage governs inference. Given input x , we compute

$$y = xA + x'(E' \odot P) + sA + kq,$$

where (A, E') is sampled uniformly from the stored (A_i, E'_i) , $k \in \mathbb{Z}^{d \times n}$ has entries drawn from the uniform distribution on

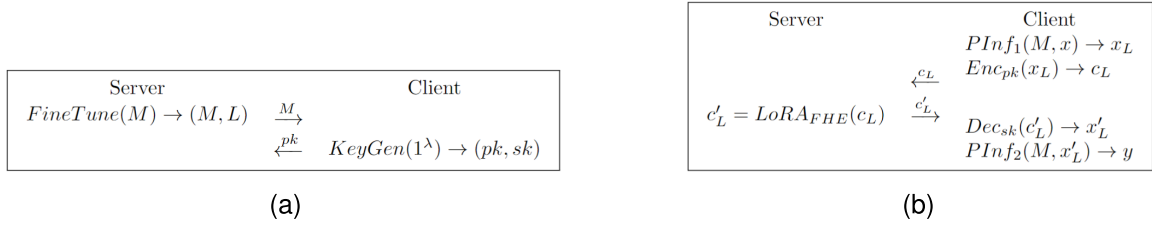


Fig. 4. "Open-LLM + Private-LoRA" structure. (a) Initialization process. (b) Inference process.

$[-q], [q]$, $P \in \{0, 1\}^{m' \times n}$ has entries drawn independently from a Bernoulli distribution with success probability p , and $\odot$ denotes the Hadamard product.

Observe that the additional term $E = x'(E' \odot P)$ acts as a random variable with an approximately Gaussian distribution but preserves the original model's performance. The intuition is as follows:

Because every entry of x' is 1 and P is Bernoulli, we have $E = x'(E' \odot P) \in \mathbb{R}^{d \times n}$. Let $e_{i,j}$ denote the (i, j) entry of E and $e'_{i,j}$ denote the corresponding entry of E' . Then

$$e_{i,j} = \sum_{i=1}^m p_{i,j} e'_{i,j},$$

where $p_{i,j}$ is Bernoulli and each $e'_{i,j}$ is fixed. The products $p_{i,j} e'_{i,j}$ are independent and bounded random variables; therefore, by Hoeffding's inequality, their sum $e_{i,j}$ is well approximated by a Gaussian distribution.

Theorem 1 (Hoeffding's Inequality): Let $X_1, \dots, X_n$ be independent random variables satisfying $a_i \leq X_i \leq b_i$. For the partial sum $S_n = \sum_{i=1}^n X_i$, we have $\forall t > 0$, $P(S_n - E[S_n] \geq t) \leq e^{-\frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2}}$.

In our setting, the random choices of $p_{i,j}$ make $\chi_{i,j} = p_{i,j} e'_{i,j}$ follow a Bernoulli distribution with $P(\chi_{i,j} = e'_{i,j}) = p$ and $P(\chi_{i,j} = 0) = 1 - p$. Because all $p_{i,j}$ are sampled independently, their sum satisfies the conditions of Hoeffding's inequality.

Introducing the random mask P does not materially degrade the network relative to the original matrix because it mirrors dropout, a standard regularization technique used to prevent overfitting [32]. During training, dropout randomly zeros a fraction of the neurons at each iteration with a fixed probability. Consequently, $E = x'(E' \odot P)$ forms a matrix whose entries are approximately Gaussian by Hoeffding's inequality, and injecting E during inference has a minimal effect on the base model's performance because of the robustness of the dropout.

Since $k \in \mathbb{Z}^{d \times n}$ is sampled uniformly over $[-q], [q]$, a deterministic result is recovered from y by

$$y = (x + s)A + E \bmod q.$$

We now analyse the construction's resistance to model extraction. For simplicity, set $d = 1$.

Definition 4 (SolveMatrix($t, m, n, q, \gamma, \beta$)): Given $t = \text{poly}(m, n)$ samples $\{(x_i \in \mathbb{R}^m, b_i = (x_i + s)A_j + e_i \bmod q)\}_{i=1, \dots, t}$, $j \sim U[1, \dots, n]$, where $s \sim \chi(\gamma)$ is a secret vector of length γ , $e_i \sim \chi(\beta)$ is a noise vector of length β , each A_j

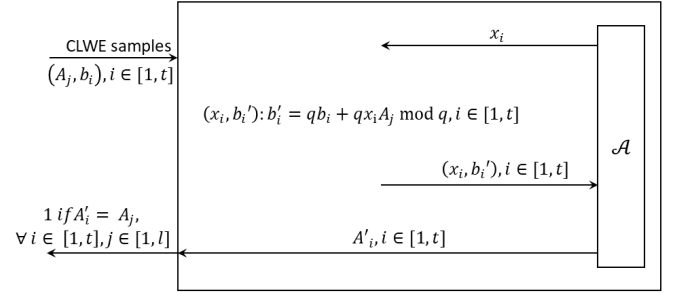


Fig. 5. Security reduction process.

has entries drawn from a standard normal distribution, and x_i is a random vector, recovering any $A_j, j \in [1, l]$.

Theorem 2: For parameters satisfying $\gamma/q \geq 2\sqrt{m}$, if an adversary can solve $SolveMatrix(t, m, n, q, \gamma, \beta)$, then it can distinguish the LWE problem, namely solve $CLWE(m, n, \gamma/q, \beta/q)$.

Proof: To prove the theorem, we transform instances of the CLWE problem into instances of $SolveMatrix(t, m, n, q, \gamma, \beta)$. The security reduction process is illustrated in Figure 5. Given the $t = \text{poly}(m, n)$ input samples of a CLWE instance, Algorithm 1 outputs the corresponding $SolveMatrix$ samples. We then feed the returned pairs (x_i, b'_i) to the adversary to recover the matrix.

On the basis of the adversary's output, we analyse two scenarios:

- If every CLWE input sample follows the CLWE distribution, i.e., $b_i = \gamma_{clwe} \cdot s_{clwe} A_j + e_{i,clwe} \bmod 1$, first rescale the modulus from 1 to a fixed real q by setting $(\bar{A}_j, \bar{b}_i) = (qA_j \bmod q, qb_i)$. This yields $\bar{A}_j \in \mathbb{R}_q^{m \times n}$ and $\bar{b}_i = \gamma_{clwe} q s_{clwe} A_j + q e_{i,clwe} \bmod q \in \mathbb{R}_q^n$. Because the transformation is invertible, $(\bar{A}_j, \bar{b}_i)$ remains computationally indistinguishable from uniform modulo q given CLWE hardness. Next, the adversary selects $x'_i \in \mathbb{R}^m$, and we compute $b'_i = \bar{b}_i + x'_i \bar{A}_j \bmod q = \gamma_{clwe} q s_{clwe} A_j + q e_{i,clwe} + q x'_i A_j \bmod q$. Compared with the definition of $SolveMatrix(t, m, n, q, \gamma, \beta)$, which requires samples $\{(x_i \in \mathbb{R}^m, b_i = (x_i + s)A_j + e_i \bmod q)\}_{i=1}^t$, we map $q x'_i \mapsto x_i$, $\gamma_{clwe} q s_{clwe} \mapsto s$, and $q e_{i,clwe} \mapsto e_i$. (Since q is public and the adversary chooses x'_i , x'_i and $q x'_i$ are equivalent from the view of the adversary.) CLWE stipulates $\gamma_{clwe} \geq 2\sqrt{m}$ with s_{clwe} a length-1 secret vector; thus, under the mapping, we obtain $\gamma = \gamma_{clwe} q \|s_{clwe}\| \geq 2q\sqrt{m}$. Thus, the t CLWE samples become (x_i, b'_i) , which meet the $SolveMatrix(t, m, n, q, \gamma, \beta)$ input format. If an

Algorithm 1 Convert Instances of the CLWE Problem Into Instances of the *SolveMatrix*($t = \text{poly}(m, n), m, n, q, \gamma, \beta$) Problem

input $t = \text{poly}(m, n)$ **samples of the CLWE problem** $\{(A_j \in \mathbb{R}_1^{m \times n}, b \in \mathbb{R}_1^n)\}$
for $i \leftarrow 1$ **to** t
 The Adversary: Randomly pick $x_i \in \mathbb{R}^m$
 Compute $b'_i = qb_i + qx_i A_j \bmod q$
return $(x_i, b'_i)_{i=1, \dots, t}$

adversary can solve *SolveMatrix*, it can recover at least one A_j .

- If the CLWE input samples are uniformly distributed, then from the adversary's perspective, b_i is uniform and does not satisfy the *SolveMatrix* structure. Since A_j is Gaussian with unit variance, the probability that the adversary outputs a particular A_j is negligible.

Hence, by inspecting whether the adversary returns one of the original matrices A_j , we distinguish CLWE samples from uniform samples, yielding an algorithm that solves CLWE.

During the reduction, given t CLWE samples (A_j, b_i) with $b_i = \gamma_{clwe} s_{clwe} A_j + e_{i,clwe} \bmod 1$, $j \in [1, t]$, and $i \in [1, t]$, proceed as follows.

1. Rescale the modulus to q to obtain $(\bar{A}_j, \bar{b}_i)$ where $\bar{A}_j = qA_j \bmod q$ and $\bar{b}_i = qb_i = \gamma_{clwe} q s_{clwe} A_j + qe_{i,clwe} \bmod q$. The invertible scaling preserves the CLWE hardness.
2. Given $x'_i \in \mathbb{R}^m$ chosen by the adversary, compute $b'_i = \bar{b}_i + x'_i \bar{A}_j \bmod q = \gamma_{clwe} q s_{clwe} A_j + qe_{i,clwe} + qx'_i A_j \bmod q$. This addition is invertible given that $(\bar{A}_j, b'_i, x'_i)$, so the CLWE hardness is also retained.
3. Discard $\bar{A}_j$ and retain $(b'_i, x_i = qx'_i)$, yielding a valid *SolveMatrix* instance with the mapping $qx'_i \mapsto x_i$, $\gamma_{clwe} q s_{clwe} \mapsto s$, and $qe_{i,clwe} \mapsto e_i$.

Note that Step 3 is one-way: (i) the CLWE hardness ensures that b_i is computationally indistinguishable from uniform modulo 1; (ii) Steps 1 and 2 are invertible, so b'_i remains indistinguishable from uniform modulo q ; therefore, (b'_i, x_i) cannot feasibly be mapped back to the original (A_j, b_i) .

4. If an adversary solves *SolveMatrix*, it violates the above one-wayness by outputting a correct A_j from its support $\mathcal{R}_1^{m \times n}$.
5. Since the one-wayness relies on CLWE hardness, we can distinguish CLWE samples from uniform samples by checking whether a valid A_j is returned.

This concludes the proof of Theorem 2. $\square$

Notably, the theorem requires $\gamma/q \geq 2\sqrt{m}$. To preserve the functionality of the privatized linear layer, γ and q must also allow the retrained network to converge. Because convergence is task dependent, suitable γ and q may not exist for every application. Nevertheless, the reduction still holds for a CLWE variant with $\gamma_{clwe} \geq \alpha\sqrt{m}$ for some $\alpha \in (0, 2)$, although the hardness of this variant remains an open problem.

C. PLL Implementation via Fhe

With the PLL in place, the server-side weights remain protected from disclosure. To further secure user inputs, we perform the PLL computations—specifically the LoRA rank-decomposition steps—using a leveled fully homomorphic scheme. Because packed encodings in leveled FHE apply each operation to every element of the ciphertext vector, we must translate the LoRA plaintext matrix operations into equivalent ciphertext-vector procedures. Recall that the computation $y = xA + x'(E' \odot P) + sA + kq$, where $x \in \mathbb{R}^{d \times m}$, $A \in \mathbb{R}^{m \times n}$, and $q \in \mathbb{R}$. The workflow is as follows:

Given an input matrix x , LoRA factorizes A as $A = A_1 A_2$ with $A_1 \in \mathbb{R}^{m \times r}$ and $A_2 \in \mathbb{R}^{r \times n}$. The low-rank matrices A_1 and A_2 are constructed as follows:

$$x = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{d1} & x_{d2} & \cdots & x_{dm} \end{bmatrix},$$

$$A_1 = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1r} \\ a_{21} & a_{22} & \cdots & a_{2r} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mr} \end{bmatrix},$$

$$A_2 = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{r1} & b_{r2} & \cdots & b_{rn} \end{bmatrix}.$$

In Section III-B, $x' \in \mathbb{R}^{d \times m'}$, $E' \in \mathbb{R}^{m' \times n}$, and $s \in \mathbb{R}^{d \times m}$ are fixed matrices, and $P \in \{0, 1\}^{m' \times n}$, $k \in \mathbb{Z}^{d \times n}$ is a random matrix. Since $Q = x'(E' \odot P) + sA + kq$ is calculated in plaintext on the server, in this section, we assume that the randomly obtained matrix Q in round t is a plaintext matrix $Q_t = x'(E' \odot P_t) + sA + k_t q$ as follows:

$$Q_t = \begin{bmatrix} q(t)_{11} & q(t)_{12} & \cdots & q(t)_{1n} \\ q(t)_{21} & q(t)_{22} & \cdots & q(t)_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ q(t)_{d1} & q(t)_{d2} & \cdots & q(t)_{dn} \end{bmatrix}.$$

The steps of the scheme are as follows:

Step 1: The client encrypts the input matrix x and transmits the encrypted matrix ct_x to the server:

$$ct_x = \text{Enc} \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{d1} & x_{d2} & \cdots & x_{dm} \end{bmatrix}.$$

Step 2: The server receives the ciphertext input matrix ct_x , performs the ciphertext–plaintext multiplication operation with the low-rank matrix A_1 , and rescales the resulting result in p bits:

$$ct_{x1} \leftarrow \text{ReScale}(\text{Mult}(A_1; ct_x); p).$$

The following shift-and-sum operation is performed on ct_{x1} to obtain ct_{x3} , where $j = 0, 1, \dots, \log(m+1)-1$ and $*$ denotes an irrelevant parameter:

$$ct_{x3} \leftarrow Add(ct_{x1}, Rotate(ct_{x1}; 2^j)),$$

$$ct_{x3} = Enc \begin{bmatrix} \sum_{k=1}^m x_{1,k} \cdot a_{k,1} * \dots * \\ \sum_{k=1}^m x_{1,k} \cdot a_{k,2} * \dots * \\ \vdots \quad \vdots \quad \ddots \quad \vdots \\ \sum_{k=1}^m x_{1,k} \cdot a_{k,r} * \dots * \\ \vdots \quad \vdots \quad \ddots \quad \vdots \\ \sum_{k=1}^m x_{d,k} \cdot a_{k,1} * \dots * \\ \sum_{k=1}^m x_{d,k} \cdot a_{k,2} * \dots * \\ \vdots \quad \vdots \quad \ddots \quad \vdots \\ \sum_{k=1}^m x_{d,k} \cdot a_{k,r} * \dots * \end{bmatrix}.$$

Step 3: Divide the irrelevant parameters and set the plaintext matrix D :

$$D = \begin{bmatrix} 1 & \dots & 0 \\ 1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 1 & \dots & 0 \end{bmatrix}.$$

Perform the ciphertext–plaintext multiplication of ct_{x3} and D to obtain ct_{x4} :

$$ct_{x4} \leftarrow ReScale(CMult(D; ct_{x3}); p),$$

$$ct_{x4} = Enc \begin{bmatrix} \sum_{k=1}^m x_{1,k} \cdot a_{k,1} & 0 & \dots & 0 \\ \sum_{k=1}^m x_{1,k} \cdot a_{k,2} & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{k=1}^m x_{1,k} \cdot a_{k,r} & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{k=1}^m x_{d,k} \cdot a_{k,1} & 0 & \dots & 0 \\ \sum_{k=1}^m x_{d,k} \cdot a_{k,2} & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{k=1}^m x_{d,k} \cdot a_{k,r} & 0 & \dots & 0 \end{bmatrix}.$$

Replicate the inner product values to other columns to obtain ct_{x5} , where $j = 0, 1, \dots, \log(m+1)-1$:

$$ct_{x5} \leftarrow Add(ct_{x4}, Rotate(ct_{x4}; -2^j)),$$

$$ct_{x5} = Enc \begin{bmatrix} \sum_{k=1}^m x_{1,k} \cdot a_{k,1} & \sum_{k=1}^m x_{1,k} \cdot a_{k,1} & \dots & \sum_{k=1}^m x_{1,k} \cdot a_{k,1} \\ \sum_{k=1}^m x_{1,k} \cdot a_{k,2} & \sum_{k=1}^m x_{1,k} \cdot a_{k,2} & \dots & \sum_{k=1}^m x_{1,k} \cdot a_{k,2} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{k=1}^m x_{1,k} \cdot a_{k,r} & \sum_{k=1}^m x_{1,k} \cdot a_{k,r} & \dots & \sum_{k=1}^m x_{1,k} \cdot a_{k,r} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{k=1}^m x_{d,k} \cdot a_{k,1} & \sum_{k=1}^m x_{d,k} \cdot a_{k,1} & \dots & \sum_{k=1}^m x_{d,k} \cdot a_{k,1} \\ \sum_{k=1}^m x_{d,k} \cdot a_{k,2} & \sum_{k=1}^m x_{d,k} \cdot a_{k,2} & \dots & \sum_{k=1}^m x_{d,k} \cdot a_{k,2} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{k=1}^m x_{d,k} \cdot a_{k,r} & \sum_{k=1}^m x_{d,k} \cdot a_{k,r} & \dots & \sum_{k=1}^m x_{d,k} \cdot a_{k,r} \end{bmatrix}.$$

Perform a multiplication operation on the ciphertexts ct_{x5} and A_2 and rescale the resulting result by p bits:

$$ct_{x6} \leftarrow ReScale(Mult(A_2; ct_{x5}); p).$$

Sum ct_{x6} every r lines to obtain ct_{x7} , where $k = 0, 1, \dots, n-1$:

$$i = [kr + 1, kr + r], ct_{x7}[k] \leftarrow Add(ct_{x6}[i + 1], ct_{x6}[i]),$$

$$ct_{x7} = Enc \begin{bmatrix} \sum_{i=1}^r \left(\sum_{k=1}^m x_{1,k} a_{k,i} b_{i,1} \right) & \dots & \sum_{i=1}^r \left(\sum_{k=1}^m x_{1,k} a_{k,i} b_{i,n} \right) \\ \sum_{i=1}^r \left(\sum_{k=1}^m x_{2,k} a_{k,i} b_{i,1} \right) & \dots & \sum_{i=1}^r \left(\sum_{k=1}^m x_{2,k} a_{k,i} b_{i,n} \right) \\ \vdots & \ddots & \vdots \\ \sum_{i=1}^r \left(\sum_{k=1}^m x_{d,k} a_{k,i} b_{i,1} \right) & \dots & \sum_{i=1}^r \left(\sum_{k=1}^m x_{d,k} a_{k,i} b_{i,n} \right) \end{bmatrix}.$$

Step 4: Add other plaintext entries, which are sent to the client:

$$ct_{x8} \leftarrow Add(ct_{x7}, Q_t),$$

Step 5: The client decrypts the ciphertext ct_{x8} to recover the product of the input matrix x with the server-side weights and then resumes the remaining model operations.

Because the inference structure minimizes the number of parameters participating in ciphertext computation, communication dominates the runtime. Each invocation of the PLL inference procedure therefore performs three ciphertext–plaintext multiplication operations and incurs two communication rounds; thus, limiting the number of interactions is critical to efficiency.

IV. EXPERIMENTS

A. Experimental Settings

1) *Hardware Setup:* We deploy the client and server on two separate machines. Device 1 runs an Intel Xeon Platinum 8375C CPU @ 2.90 GHz with 48 GB RAM, an NVIDIA RTX A6000 GPU, and Ubuntu 22.04.3 LTS (Linux kernel 6.5.0–28-generic). Device 2 runs an Intel Xeon Gold 6133 CPU @ 2.50 GHz with 24 GB RAM, an NVIDIA RTX 4090 GPU, and Ubuntu 22.04.4 LTS (Linux kernel 6.5.0–21-generic). The network bandwidth between devices is 3 Gbps with an average round-trip time of approximately 1 ms.

TABLE I
COMPARISON OF MODEL PERFORMANCE BETWEEN THE
ORIGINAL FINE-TUNING AND OUR SCHEME

Method	Accuracy	Macro Precision	Macro Recall	Macro F1
Original Fine-tuning	98.94%	0.9886	0.9862	0.9871
Our Scheme	97.94%	0.9761	0.9783	0.9789

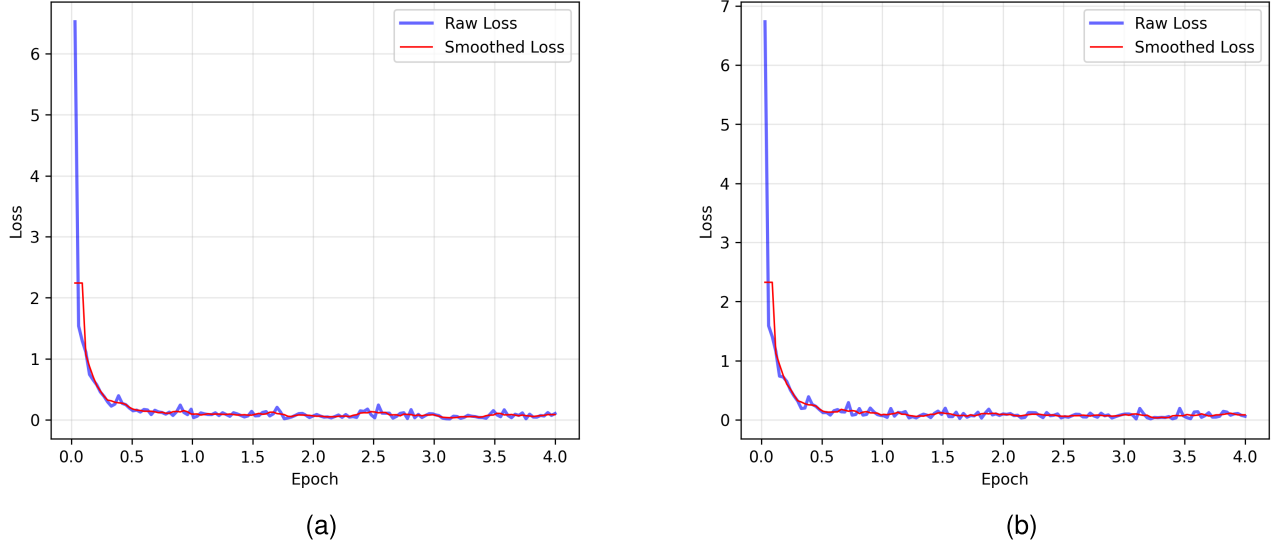


Fig. 6. Comparison of training loss curves. (a) Original fine-tuning. (b) Our method.

2) *Models and Fine-Tuning*: We use ChatGLM2-6B as the base model, a Chinese–English bilingual dialogue model released in June 2023 and open-sourced in July by Zhipu AI and Tsinghua KEG Lab [33]. ChatGLM2-6B builds on the general language model (GLM) architecture [34] with 6.2 billion parameters, offering stronger performance, faster inference, and fully open weights for academic research. We implement LoRA using LLaMA-Factory [35], injecting bypass matrices into the Q, K, and V weight matrices of the self-attention layers.

3) *Fhe Implementation*: We use the CKKS scheme. For efficiency, the implementation combines C++ and Python: Microsoft SEAL [29] for CKKS in C++, and SEAL-Python (Python bindings for SEAL) in Python. CKKS parameters are chosen to support a maximum multiplication depth of 4.

B. Improving Computation and Communication Efficiency

To accelerate ciphertext operations, we parallelize the server-side computation. Because Python’s threads are OS-native and contend for the global interpreter lock, true parallelism is not achievable in pure Python. We therefore implement the server-side algorithm in C++ with thread pooling, achieving a 20%–40% speedup in inference.

With respect to ciphertext transmission, Python’s built-in serialization can cause data loss when it is deserialized in C++. We use Protocol Buffers (protobuf) [36] for communication. Protobuf is a lightweight, efficient format for structured data serialization and is well suited for storage and RPC data exchange. This approach provides reliable serialization and deserialization, preserving data integrity and accuracy.

C. Application Scenario

Our method is applicable to many vertical domains because of the strong generalizability of the LLMs, such as sensitive data classification. Sensitive data classification is a common information security practice that assigns data to levels by importance and sensitivity and then applies appropriate protection measures to ensure confidentiality, integrity, and availability. Sensitive data include personally identifiable information, financial records, health data, trade secrets, and confidential government information. Unauthorized exposure, tampering, or access can severely impact individuals, organizations, or society. When using LLMs to process such data, privacy-preserving techniques are essential. We therefore evaluate our scheme on this application.

In this setting, the input is a message to be classified, and the output is a category label. We fine-tune the LLM with LoRA on 5,360 nonrepeated training samples and evaluate it on 283 nonrepeated test samples. We then apply our method to produce a model that classifies sensitive data while protecting both user inputs and private weights.

D. Training Efficiency

In this experiment, both the original fine-tuning method and our scheme use cross-entropy loss, the standard loss function for LoRA fine-tuning in language modelling, instruction tuning, and downstream tasks. The results are summarized in Table I. The original fine-tuning achieves 98.94% accuracy with a macro precision of 0.9886, a macro recall of 0.9862, and a macro F1 of 0.9871. Our method achieves 97.94% accuracy with a macro precision of 0.9761, a macro recall of

0.9783, and a macro F1 of 0.9789, while the weighted metrics remain stable at approximately 0.97. These results show that the decrease in accuracy is marginal and acceptable.

Both methods converge within 4 epochs. The original method reaches a final loss of 0.1036, while that of our method reaches 0.0616. The overall convergence trend is clear, and the epoch-level averaged curves confirm the stability of our method. The main difference is the training time: the baseline method completes in 17 minutes, whereas our method requires 37 minutes, an increase of approximately $2.1\times$, which remains acceptable for practical use. This increase stems primarily from additional computations introduced by the privacy-preserving mechanism, not from convergence difficulties.

Figure 6 compares the training loss curves of the original fine-tuning method and our method. Both plots show training loss on the original scale. The blue lines represent the raw loss values; the red lines are smoothed to highlight the overall convergence trend. Both methods converge effectively within 4 epochs, indicating that our method does not impair training stability.

For both methods, loss decreases rapidly in the first few epochs and stabilizes near zero, demonstrating effective convergence. The logarithmic scale confirms that both methods achieve consistent reductions in loss values, even at very small magnitudes, without oscillations or divergence.

A comparison of the two sets of curves reveals that our privacy-preserving method does not negatively affect convergence. Both approaches exhibit similar training dynamics, indicating that the observed increase in training time stems from additional privacy-related computations rather than optimization difficulties.

Overall, the experimental evidence demonstrates that our method preserves model functionality while providing privacy protection. The performance degradation is within a tolerable range, and the extra training cost is explained by the privacy mechanism itself, not by instability in the fine-tuning process.

E. Performance Across Tasks

To assess performance variance across tasks, we evaluate two classification datasets: a binary sentiment analysis task and a 32-class news topic classification task. For each task, we compare the original LoRA model against our PLL-Model to quantify the impact of the privacy-preserving transformation.

The results in Figure 7 and Figure 8 show that our method maintains high performance with minimal overhead.

- **Sentiment Analysis:** Our PLL-Model achieves an F1 score of 0.9182, a decrease of only **1.2%** compared with the original model's F1 score of 0.9296.
- **32-class News Topic Classification:** In this more challenging task, our model achieves an F1 score of 0.8003, a decrease of just **0.9%** from the original model's F1 score of 0.8077.

These results yield two insights:

- 1) **Low-Performance Impact:** The degradation from injected noise and randomized dropout is consistently

Performance Comparison on Sentiment Classification Task

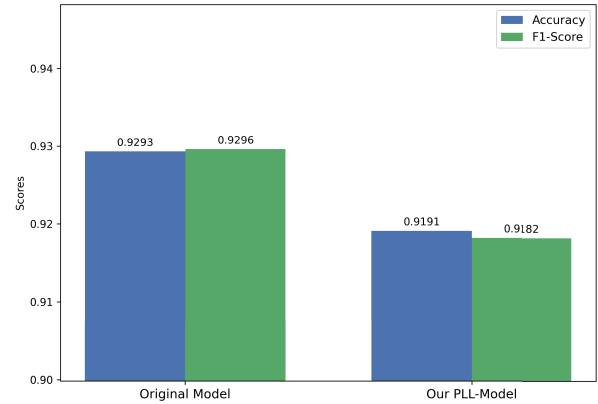


Fig. 7. Performance comparison on the sentiment classification task. The performance drop of our PLL model is minimal, demonstrating effective utility preservation.

Performance Comparison on News Topic Classification Task

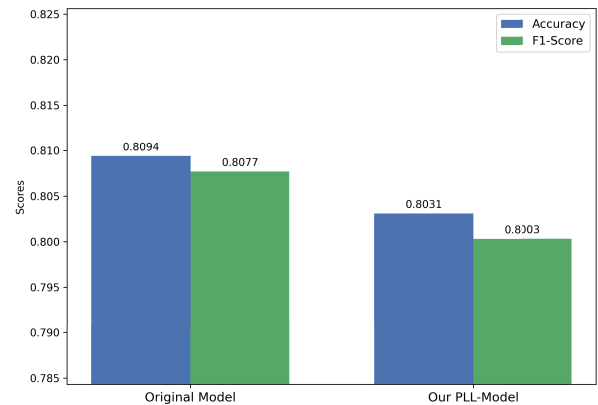


Fig. 8. Performance comparison on the 32-class news topic classification task. Our model maintains high performance, showing strong generalizability and low impact from the privacy transformation.

minimal (approximately 1%), confirming that our method preserves model utility.

- 2) **Strong Generalization:** Maintaining high performance across structurally different classification tasks results in strong generalization and low variance.

This analysis quantitatively shows that our method enhances security without significantly compromising practical effectiveness across diverse tasks.

F. Inference Speed

Because CKKS supports single-instruction multiple data (SIMD) [37], homomorphic addition and multiplication run in parallel, making our method well suited for inputs with many tokens. With the LoRA rank set to 8, the performance converges to 1.61 seconds per token when the token count reaches 1,000. The run times for varying token counts and the corresponding communication costs are shown in Figure 9. Notably, communication accounts for only a small fraction of the total latency, increasing slightly from 2.37% to 3.14% as the token count scales from 50 to 1,000, indicating that

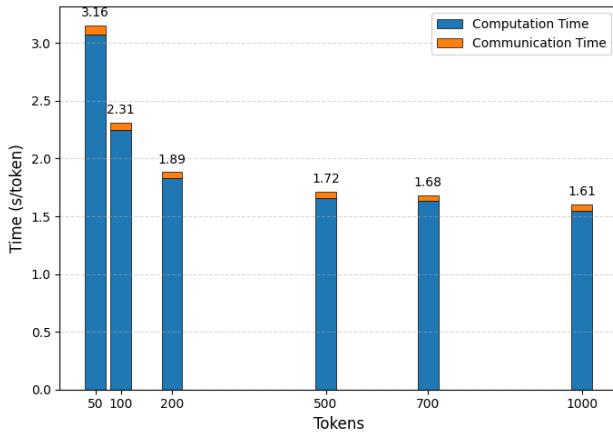


Fig. 9. Run times and communication overhead under different numbers of tokens (LoRA rank = 8).

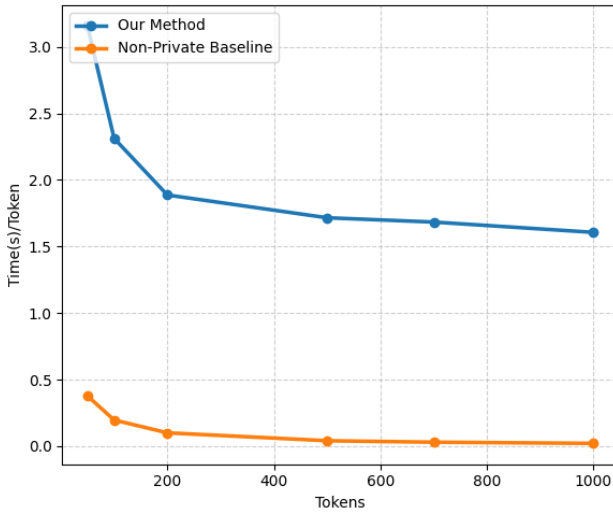


Fig. 10. Inference latency of the proposed method and the nonprivate baseline under different numbers of tokens (LoRA rank = 8).

communication remains a secondary factor even at larger input sizes.

Figure 10 compares the inference efficiency of our method with that of the baseline ChatGLM2-6B+LoRA architecture without privacy protection. Our privacy-preserving framework incurs a latency overhead of approximately $8.4\times$ to $76\times$ compared with plaintext inference, depending on the input size. This overhead stems from homomorphic encryption computations, private FHE operations, and transmission costs due to ciphertext expansion.

The efficiency of processing 500 tokens with different LoRA ranks is shown in Figure 11. Efficiency scales approximately linearly with rank, indicating that our method extends to linear layers with larger dimensions. This suggests potential applicability to fine-tuned LLMs where $r = d$.

In ultralow latency scenarios, inference performance is minimally affected since communication accounts for only 2%–3% of the total overhead. However, for extremely large-scale models, the increasing size of the transformer blocks increases the number of parameters stored in the LoRA

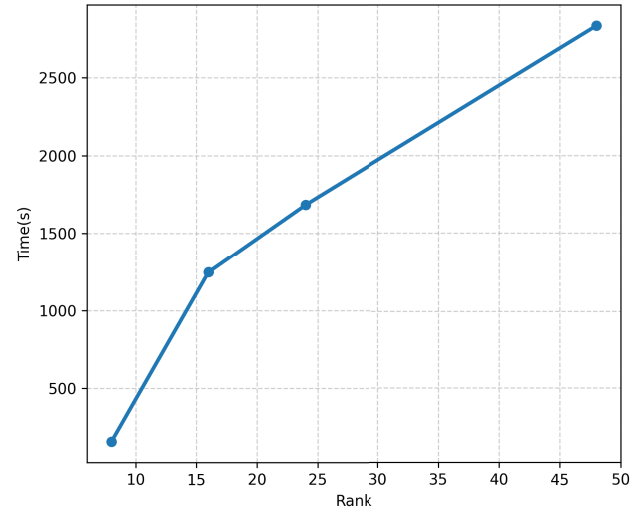


Fig. 11. Run time of 500 tokens with different ranks.

TABLE II
PRIVACY INFERENCE ALGORITHMS IMPLEMENTED
ON SMALLER LLMs

Year	Schemes	Model	#Params	Hardware Set.	Network Set.	Comm.	Time(s)/Token
2022	THE-X [6]	Bert-tiny	<14.5M	-	-	-	-
2022	Iron [7]	BERT-Base	110M	Intel Xeon 8000 series, 3.6 GHz	(3Gbps, 0.8ms)	280.99GB	475s
2022	ConcreteML [10]	GPT-2	117M	-	-	-	107.87s
2023	CipherGPT [9]	GPT-2	117M	Intel Xeon 8000 series, 3.6 GHz	(3Gbps, 0.8ms)	14.74GB	1180.93s
2023	BumbleBee [8]	GPT-2	117M	Intel Xeon 8000 series	(1Gbps, 0.5ms)	6.61GB	183.6s
2023	PUMA [15]	GPT-2	117M	Intel Xeon Platinum 8369B (Ice Lake), 32 vCPUs @ 2.70 GHz	(5Gbps, 1ms)	3.774GB	15.5s
2024	SecureGPT [12]	GPT-2	117M	Intel Xeon 8000 series, 3.6 GHz	(1Gbps, 0.1ms)	-	37.52s
2024	Rovida et al. [13]	BERT-tiny	<14.5M	Apple M1 Pro, 10 cores @ 3.20 GHz	-	-	$\approx 15s$

TABLE III
PRIVACY INFERENCE ALGORITHMS IMPLEMENTED ON LLMs
WITH BILLION+ PARAMETER SIZES

Year	Schemes	Model	#Params	Hardware Set.	Network Set.	Comm.	Time(s)/Token
2023	BumbleBee [8]	LLaMA-7B	7B	Intel Xeon Scalable (Ice Lake), 64 vCPUs @ 2.70 GHz	(1Gbps, 0.5ms)	5.64GB	832.2s
2023	PUMA [15]	LLaMA-7B	7B	Intel Xeon Scalable (Ice Lake), 128 vCPUs @ 2.70 GHz	(20Gbps, 0.1ms)	1.794GB	200s
2025	Ours	ChatGLM2	6B	Intel (R) Xeon (R) Platinum 8375C CPU @ 2.90GHz	(3Gbps, 1ms)	0.569GB	1.61s

layers and leads to linear growth in computational demand, constraining the scalability of our scheme. Future work may explore integrating model quantization, pruning, or dedicated hardware accelerators to reduce computational latency and enhance applicability across broader scenarios.

In Tables II and III, we list the privacy inference algorithms for the LLMs proposed in recent years, with the parameters of the LLMs and their efficiency. “-” indicates that it is not explicitly stated in the original paper. We can see that the earlier proposed scheme is not suitable for larger models. For large models with more than a billion parameters, PUMA is the most efficient method. The comparison shows that our scheme takes approximately 1.61 s to generate 1 token in the inference process, which is 127 times faster than PUMA.

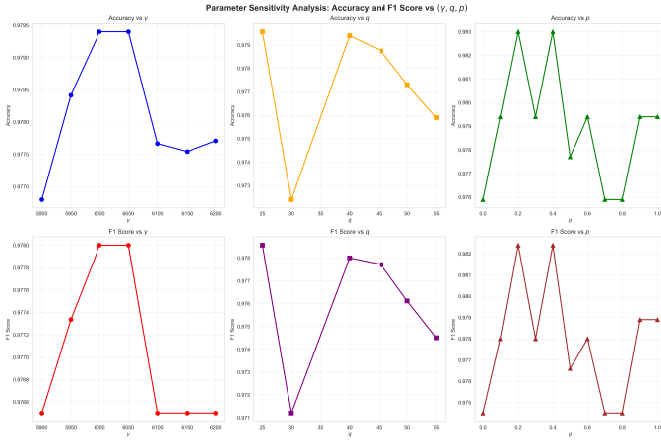


Fig. 12. Sensitivity analysis of the F1 score and accuracy with respect to the hyperparameters (γ, q, p). The minimal fluctuations in performance across wide parameter ranges demonstrate the model's high robustness.

G. Security Discussion

A subtle but important point is that a privacy-preserving inference model using a single privacy-preserving technology (e.g., FHE or MPC) may protect inputs but not model weights. Considering MPC, as noted by Yehuda Lindell, a leading expert in MPC [38], “Another subtlety that is often misunderstood is that MPC secures the process, meaning that nothing is revealed by the computation itself. However, this does not mean that the output of the function being computed does not reveal sensitive information. For an extreme example, consider two people computing the average of their salaries. It is indeed true that nothing but the average will be output, but given a person's own salary and the average of both salaries, they can derive the exact salary of the other person. Thus, just using MPC does not mean that all privacy concerns are solved.” Further evidence comes from a best paper at Eurocrypt 2025 [39], which presented techniques to extract DNN parameters using a polynomial number of queries in polynomial time. The paper also shows that all weights of a ReLU-based DNN can be efficiently determined by analysing the geometric shape of its decision boundaries. Therefore, a privacy-preserving inference model that uses only FHE or MPC during inference without modifying model weights may be vulnerable to cryptanalytic extraction attacks because the model preserves the original input–output relationships that enable such attacks.

Consequently, these model extraction attacks may apply to privacy-preserving inference models that rely solely on FHE [6], [13] or MPC [9], [15], [16].

In contrast, our scheme provides bidirectional protection: The model weights remain encrypted throughout inference and are never exposed to the user, while input data are kept private from the server. To our knowledge, our method is the first to provide a practical approach with theoretical guarantees for protecting model weights in privacy-preserving inference.

H. Parameter Sensitivity and Robustness Analysis

We evaluate robustness through a sensitivity analysis of three key hyperparameters (γ, q, p). Each parameter was varied

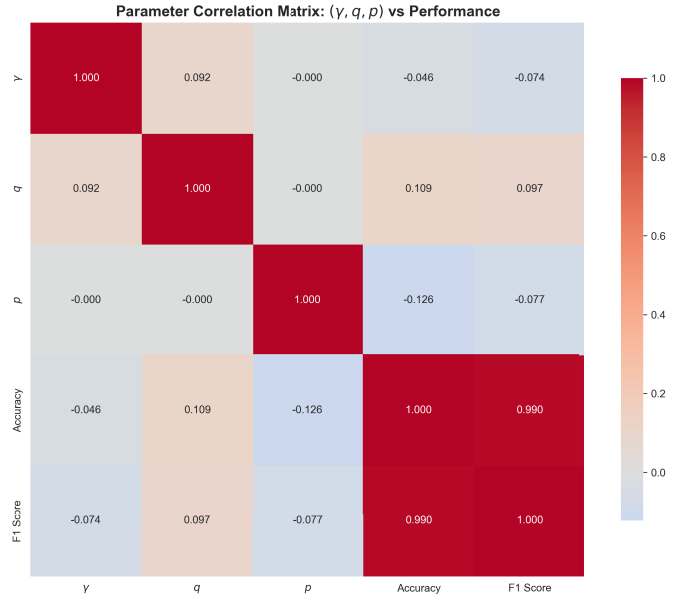


Fig. 13. Correlation matrix of hyperparameters and performance metrics. The low correlation values (all absolute values < 0.15) statistically confirm the model's low sensitivity to the tested parameters.

systematically within a predefined range while holding the others constant, and we measured the effects on the F1 score and accuracy on the test set.

The results in Figure 12 show that our model exhibits low sensitivity to these parameters. Specifically, across the entire tested range, we find the following:

- **Minimal Performance Fluctuation:** The variations in the F1 score and accuracy were consistently small, with the total fluctuation range not exceeding 1.3 percentage points for any parameter.
- **Low Correlation:** The correlation matrix in Figure 13 confirms this finding. The absolute values of the correlation coefficients between all three hyperparameters and the performance metrics (F1 and accuracy) are less than 0.15, indicating a lack of a significant linear relationship.

This outcome demonstrates the **robustness** of our method. A model that is insensitive to hyperparameter changes is highly desirable in practice, as it implies stable and reliable performance across diverse settings without costly fine-grained tuning. Our analysis confirms that our model is effective, robust, and stable.

V. CONCLUSION

For the paradigm of fine-tuning pretrained LLMs to improve performance in specialized domains, we combine privacy-preserving techniques, including FHE and provable security theory, to propose an efficient and secure ciphertext inference scheme for LLMs that protects both the user inputs and the server-side private training data. Our method separates the low-rank matrices obtained via LoRA fine-tuning from the pretrained model and performs private computations only on the former to reduce the overhead. We also introduce a general method to transform a linear layer into a PLL

that preserves the original functionality and provides security against model extraction attacks. Using these techniques, our scheme achieves an inference latency of less than 1.61 seconds per token when the token count exceeds 1,000, making it a practical privacy-preserving LLM solution.

REFERENCES

- [1] N. Carlini, C. Liu, U. Erlingsson, J. Kos, and D. Song, "The secret sharer: Evaluating and testing unintended memorization in neural networks," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 267–284.
- [2] N. Carlini et al., "Extracting training data from large language models," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2633–2650.
- [3] OpenAI.(2023). *Privacy Policy*. [Online]. Available: <https://openai.com/policies/privacy-policy>
- [4] A. Al Badawi et al., "Towards the AlexNet moment for homomorphic encryption: HCNN, the first homomorphic CNN on encrypted data with GPUs," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 3, pp. 1330–1343, Jul. 2021.
- [5] A. Vaswani et al., "Attention is all you need," in *Proc. NeurIPS*, vol. 30, Jan. 2017, pp. 5998–6008.
- [6] T. Chen et al., "The-x: Privacy-preserving transformer inference with homomorphic encryption," 2022, *arXiv:2206.00216*.
- [7] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang, "Iron: Private inference on transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 15718–15731.
- [8] W. Lu et al., "BumbleBee: Secure two-party inference framework for large transformers," *Cryptol. ePrint Arch.*, Paper 2023/1678, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1678>
- [9] X. Hou et al., "CIPHERGPT: Secure two-party gpt inference," *Cryptol. ePrint Arch.*, vol. 2023, no. 372, pp. 1–15, Oct. 2023.
- [10] A. Meyre et al., "Concrete ML: A privacy-preserving machine learning library using fully homomorphic encryption for data scientists," GitHub Repository, 2022. [Online]. Available: <https://github.com/zamaai/concrete-ml>
- [11] M. Gong et al., "A multi-modal vertical federated learning framework based on homomorphic encryption," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1826–1839, 2024.
- [12] C. Zeng, D. He, Q. Feng, X. Yang, and Q. Luo, "SecureGPT: A framework for multi-party privacy-preserving transformer inference in GPT," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 9480–9493, 2024.
- [13] L. Rovida and A. Leporati, "Transformer-based language models and homomorphic encryption: An intersection with BERT-tiny," in *Proc. 10th ACM Int. Workshop Secur. Privacy Anal.*, 2024, pp. 3–13.
- [14] J. Zhao, K. Chen, X. Yuan, Y. Qi, W. Zhang, and N. Yu, "Silent guardian: Protecting text from malicious exploitation by large language models," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 8600–8615, 2024.
- [15] Y. Dong et al., "PUMA: Secure inference of LLaMA-7B in five minutes," 2023, *arXiv:2307.12533*.
- [16] D. Rathee, D. Li, I. Stoica, H. Zhang, and R. Popa, "MPC-minimized secure LLM inference," 2024, *arXiv:2408.03561*.
- [17] K.-Y. Lam, X. Lu, L. Zhang, X. Wang, H. Wang, and S. Goh, "Efficient fhe-based privacy-enhanced neural network for AI-as-a-service," *Cryptol. ePrint Arch.*, vol. 2023, no. 503, pp. 1–20, Jul. 2023.
- [18] J. E. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2021, p. 3.
- [19] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," in *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 10088–10115.
- [20] A. N. Lee, C. J. Hunter, and N. Ruiz, "Platypus: Quick, cheap, and powerful refinement of LLMs," 2023, *arXiv:2308.07317*.
- [21] J. M. Z. Chaves et al., "TX-LLM: A large language model for therapeutics," 2024, *arXiv:2406.06316*.
- [22] X. Lisa Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," 2021, *arXiv:2101.00190*.
- [23] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," 2021, *arXiv:2104.08691*.
- [24] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 3104–3112.
- [25] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder–decoder approaches," 2014, *arXiv:1409.1259*.
- [26] R. Rivest, "On databanks and privacy homomorphism," in *Foundations of Secure Computation*. New York, NY, USA: Academic, 1978.
- [27] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. Annu. ACM Symp. Theory Comput.*, May 2009, pp. 169–178.
- [28] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2017, pp. 409–437.
- [29] (Jan. 2023). *Microsoft SEAL (Release 4.1)*. [Online]. Available: <https://github.com/Microsoft/SEAL>
- [30] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," in *Proc. Annu. ACM Symp. Theory Comput.*, May 2005, pp. 84–93.
- [31] J. Bruna, O. Regev, M. J. Song, and Y. Tang, "Continuous LWE," in *Proc. 53rd Annu. ACM SIGACT Symp. Theory Comput.*, Jun. 2021, pp. 694–707.
- [32] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," 2012, *arXiv:1207.0580*.
- [33] (2023). *ChatGLM2-6b: An Open Bilingual Chat LLM*. [Online]. Available: <https://github.com/zai-org/ChatGLM2-6B>
- [34] Z. Du et al., "GLM: General language model pretraining with autoregressive blank infilling," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics*, vol. 1, 2022, pp. 320–335.
- [35] Y. Zheng et al., "LlamaFactory: Unified efficient fine-tuning of 100+ language models," 2024, *arXiv:2403.13372*.
- [36] K. Varda. (2008). *Protocol Buffers: Google's Data Interchange Format*. [Online]. Available: <https://opensource.googleblog.com/2008/07/protocol-buffers-googles-data.html>
- [37] N. P. Smart and F. Vercauteren, "Fully homomorphic SIMD operations," *Designs, Codes Cryptography*, vol. 71, no. 1, pp. 57–81, Apr. 2014.
- [38] Y. Lindell, "Secure multiparty computation," *Commun. ACM*, vol. 64, no. 1, pp. 86–96, 2020.
- [39] N. Carlini, J. Chávez-Saab, A. Hambitzer, F. Rodríguez-Henríquez, and A. Shamir, "Polynomial time cryptanalytic extraction of deep neural networks in the hard-label setting," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2025, pp. 364–396.